

餘燼啟示錄：開發建議

諮詢問題：現在是HarpoonTurret有錯誤

作為一位資深的遊戲開發架構師，我仔細分析了您提供的《餘燼航路：鏽蝕重生》開發日誌、企劃內容以及各個 GDScript 檔案。綜合來看，整個專案展現了高度的創意和工程潛力，尤其是在遊戲機制和世界觀設定上。

然而，根據您提出的`HarpoonTurret`錯誤報告，以及我對相關腳本的深入審查，我發現了幾個核心的設計衝突、API 調用錯誤以及數據流不一致的問題。這些問題導致了`HarpoonTurret`在執行捕獲、拉扯和蒸汽消耗等關鍵邏輯時出現故障。

以下是我的分析、診斷及具體的改造建議：

核心問題分析：`HarpoonTurret`的多重邏輯斷層

`HarpoonTurret`作為船隻戰鬥的核心組件，其設計理念與周邊實體（`HarpoonProjectile`、`Monster`、`Global`/`ShipManager`）的通信契約存在嚴重脫節。

1. ****HarpoonProjectile (`harpoon.gd`) 與 HarpoonTurret (`harpoon\_turret.gd`) 之間的通信契約不一致：****

- * `HarpoonTurret` 試圖連接 `harpoon.gd` 中不存在的 `retracted\_to\_launcher` 信號。
- * `HarpoonTurret` 調用 `\_current\_harpoon.get\_attached\_target()`，但 `HarpoonProjectile` 內部變數 `\_target\_body` 是私有的，且沒有提供公共的 getter 方法。
- * `HarpoonTurret` 調用 `\_current\_harpoon.request\_retraction\_complete()`，但 `HarpoonProjectile` 中並無此方法，其回歸邏輯是透過 `launcher.complete\_retraction()` 直接調用 `HarpoonTurret` 的方法。這種單向調用缺乏雙向溝通的彈性。

2. ****HarpoonTurret 與 Monster (`monster.gd`) 之間的交互邏輯混亂 : ****

* `HarpoonTurret` 試圖調用 `monster\_target.apply\_pull\_force()` 和 `target.get\_trapped()` , 這些方法在 `monster.gd` 中並不存在。

* `monster.gd` 內部有 `\_process\_hooked\_logic` , 其中包含了 `Global.apply\_steam\_pressure\_loss()` 。這意味著怪物自己在被勾住時會消耗蒸汽, 與砲台的蒸汽消耗邏輯重疊, 且 `Global.gd` 中並無 `apply\_steam\_pressure\_loss` 方法。

3. ****Steam 資源管理與 Singleton 調用混淆 : ****

* `harpoon\_turret.gd` 內部緩存了一個 `ship\_manager` 節點, 並調用其 `can\_afford\_steam()` 和 `request\_steam\_consumption()` 。然而, `ship\_manager.gd` 腳本提供的 `steam\_pressure` 和 `max\_pressure` 屬性與 `Global.gd` 中的 `ship\_stats.steam\_pressure` 和 `max\_pressure` 存在 ****重複且不一致**** 。`Global.gd` 才是被標記為「全域數據中心 v2.6 - 鋼化版」的核心單例, 實際的蒸汽扣除邏輯 (`apply\_steam\_loss`) 也位於 `Global.gd` 中。這種多個單例負責同類數據的管理方式極易引發 Bug。

4. ****UI 交互與物品持有狀態判斷 : ****

* 在 `player.gd` 中, `\_find\_cabinet\_target` 在獲取 `available\_items` 時, 雖然 `cabinet.gd` 有 `@export var available\_items: Array[String]` , 但直接 `cabinet.get("available\_items")` 可能返回 `null` , 如果 `items.size() > 0` 沒有防禦性檢查, 也會有潛在問題 (已在 `player.gd` 腳本中做了鋼化修正) 。

診斷與改造建議 : 建立清晰的組件契約

為了解決上述問題, 我將從架構層面重構各組件的職責, 並提供具體的程式碼修改方案。

一、架構原則 : 統一數據源與清晰的通信契約

1. ****單一事實來源 (Single Source of Truth) : **** 統一由 `Global.gd` 管理所有船隻核心數據 (如蒸汽壓力) , 淘汰 `ShipManager.gd` 中的重複數據, 或將 `ShipManager.gd` 改造為 `Global.gd` 的一個輔助模組, 只提供與船隻運營相

關的計算服務，而不持有獨立的數據。

2. ****明確的接口 (Clear Interfaces):**** 定義每個組件對外提供的公共方法和信號，確保調用方只使用這些明確的接口，而非直接訪問私有變數或假定方法存在。
3. ****事件驅動 (Event-Driven):**** 關鍵狀態變化（如蒸汽消耗、怪物被捕獲）應透過信號發布，而非直接調用，以減少組件間的強耦合。

二、具體程式碼改造方案

改造目標 1：統一 Steam 壓力管理（淘汰 `ShipManager.gd` 的重複職責）

* ****`ship\_manager.gd` (文件):****

* ****行動:**** 考慮將其更名為 `ShipStatsHelper.gd` 或直接刪除其 AutoLoad 設置，並將其邏輯合併到 `Global.gd` 或相關組件中。****** 由於 `Global.gd` 已經是「全域數據中心 v2.6 - 鋼化版」，且包含了 `ship\_stats`，`ShipManager.gd` 現有的 `steam\_pressure` 和 `max\_pressure` 屬性是冗餘的，會導致數據不一致。

* ****建議:**** 如果 `ShipManager.gd` 有其他不可替代的職責，請明確。目前來看，它的核心數值可以完全由 `Global.gd` 管理。

* ****`Global.gd` (文件):****

* ****行動:**** 確保 `update\_steam(amount)` 和 `apply\_steam\_loss(loss)` 是唯一修改蒸汽壓力的公共接口。

* ****新增:**** `can\_afford\_steam(cost: float) -> bool`

``gdscript

Global.gd 新增方法

func can_afford_steam(cost: float) -> bool:

 return ship_stats.steam_pressure >= cost

...

改造目標 2：HarpoonProjectile 與 HarpoonTurret 的通信契約

* ****`harpoon.gd` (HarpoonProjectile 類):****

* ****新增信號:****

``gdscript

harpoon.gd

signal retracted_to_launcher(harpoon_instance:

HarpoonProjectile) # 當魚叉完全被收回時發出

```
...
* **新增公共 getter 方法：** 允許砲台獲取魚叉鉤住的目標。
  `` `gdscript
  # harpoon.gd
  func get_attached_target_body() -> Node2D:
    return _target_body
...

* **調整回收邏輯：** 當魚叉被收回並準備 `queue_free()` 時，發出信號。
  `` `gdscript
  # harpoon.gd _physics_process 中的 _is_pulled 區塊
  # ...
  if global_position.distance_to(launcher.global_position) < 50.0:
    if launcher: # 確保發射器存在
      retracted_to_launcher.emit(self) # 發送信號
      queue_free() # 刪除子彈實體
    return
...

* **移除 `request_retraction_complete()`：** 此方法不再需要，因為砲台
將透過信號接收。

* **`harpoon_turret.gd` (文件):**
  * **移除 `_ship_manager` 引用與相關調用：** 將所有
  `_ship_manager.can_afford_steam` 和
  `_ship_manager.request_steam_consumption` 替換為
  `Global.can_afford_steam` 和 `Global.apply_steam_loss`。
  * **修改信號連接：**
    `` `gdscript
    # harpoon_turret.gd _fire 方法中
    # ...
    # if is_instance_valid(p) and
    p.has_signal("retracted_to_launcher"): # 舊代碼
      # p.retracted_to_launcher.connect(complete_retraction)

    # 新代碼：直接連接到魚叉的信號
    if is_instance_valid(p):
      p.retracted_to_launcher.connect(complete_retraction)
```

```

        p.launcher = self # 建立連結
    # ...
    ...

* **修改 `_handle_retraction_logic` : **
    * 將 `_current_harpoon.get_attached_target()` 替換為
    `_current_harpoon.get_attached_target_body()`。
    * 蒸汽消耗改為調用 `Global` :
        ```gdscript
 # harpoon_turret.gd _handle_retraction_logic 方法中
 # ...
 if pull_force > 0 and Global.can_afford_steam(steam_cost): #
改用 Global
 Global.apply_steam_loss(steam_cost) # 改用 Global 的扣除方
法
 # ...
 ...

* **拉力應用方式：** 砲台直接對怪物施加力，而不是讓怪物自行處理拉力。
    ```gdscript
    # harpoon_turret.gd _handle_retraction_logic 方法中
    # ...
    # 讓目標自行處理受力 (這是舊邏輯，假設怪物有 apply_pull_force)
    # monster_target.apply_pull_force(pull_dir * (pull_force / (1.0
+ mass/100.0)) * delta)

    # 新邏輯：砲台直接影響怪物的 velocity
    var pull_vec = pull_dir * (pull_force / (1.0 + mass/100.0)) *
delta
    monster_target.velocity += pull_vec # 直接疊加到怪物的 velocity
    # ...
    ...

* 移除 `_current_harpoon.request_retraction_complete()` 的所有調
用，因為現在是由魚叉自己發出信號。

* **修改 `complete_retraction` : ** 接收魚叉實例。
    ```gdscript
 # harpoon_turret.gd complete_retraction 方法
 # 移除 `retracted_harpoon: HarpoonProjectile` 參數，因為信號中已經帶

```

有 `self`

```
func complete_retraction(retracted_harpoon_instance:
HarpoonProjectile):
 if is_instance_valid(_current_harpoon) and _current_harpoon !
= retracted_harpoon_instance:
 return # 確保是我們當前發射的魚叉

 _current_harpoon = null
 _set_turret_state(TurretState.RELOADING) # 進入裝填狀態，會自
動處理視覺和切回 IDLE
 harpoon_reloaded.emit()
...

```

\*注意：在 `harpoon.gd` 的 `\_is\_pulled` 區塊中，  
`launcher.complete\_retraction()` 調用應修改為  
`launcher.complete\_retraction(self)`，以匹配新的信號簽名。\*

#### 改造目標 3：Monster 的交互接口與邏輯簡化

```
* **`monster.gd` (文件):**
* **移除冗餘蒸汽消耗：**
 ``gdscript
 # monster.gd _process_hooked_logic 方法中
 # Global.apply_steam_pressure_loss(cfg.drag * delta) # 移除此
行！
...

```

這部分蒸汽消耗將完全由 `HarpoonTurret` 在拉扯時計算並向 `Global` 請求扣除。

```
* **新增公共方法：**
 ``gdscript
 # monster.gd
 func get_weight() -> float:
 # 根據怪物的類型或配置返回其重量，用於計算拉力阻力
 # 這裡可以根據 cfg 字典或怪物自身的屬性來定義
 return 100.0 # 示例值

```

```
新增方法，用於在被拉到甲板時觸發，取代 get_trapped()
func on_pulled_to_deck():

```

```

 is_on_ship = true
 is_hooked = false
 velocity = Vector2.ZERO # 清零速度，防止滑動
觸發其他上甲板動畫或效果
 print("⚓ [系統] 怪物成功被拖拽至甲板，進入戰鬥準備。")

確保有設定 hooked 狀態的方法
 func set_is_hooked(state: bool):
 is_hooked = state

確保有設定 ready_for_dismantle 狀態的方法
 func prepare_for_dismantle():
 # 此處假設 monster 腳本有 is_ready_for_dismantle 屬性
 # 或者怪物死亡後可以設定一個狀態，讓玩家可以拆解
 set("is_ready_for_dismantle", true) # 或者直接將
is_ready_for_dismantle 設為 public var
 print("⚓ [系統] 怪物已準備好進行拆解。")
 ...

* **調整`_process_hooked_logic`:** 此邏輯應僅處理怪物自身被鉤住時的
掙扎（而不是拉力），拉力由砲台施加。
` ``gdscript
monster.gd _process_hooked_logic 方法
func _process_hooked_logic(delta: float, cfg: Dictionary):
 # 這裡只處理怪物自身的掙扎反作用力，而不是拉力本身
 # 砲台在 _handle_retraction_logic 中會直接修改怪物的 velocity
 # 為了模擬掙扎，我們可以讓怪物嘗試朝反方向移動或輕微抖動
 if Global.ship_instance:
 var dir =
global_position.direction_to(Global.ship_instance.global_position)
 # 模擬掙扎：讓怪物產生一個與拉力方向相反的輕微推力
 velocity += -dir * cfg.drag * 50.0 * delta # cfg.drag 越大掙扎
越強

 # 讓怪物維持在空中或海中，直到被拉上船
 # （這裡的 Y 軸邏輯需要根據你的怪物類型和行為仔細調整）
 # 簡單起見，在被鉤住時，可能限制其 Y 軸移動或讓其輕微浮動

由於HarpoonTurret會直接修改velocity，此處的掙扎邏輯可能會被覆

```

蓋

# 更好的方式是，如果HarpoonTurret已經施加了強力拉扯，這裡的掙扎力可以被削弱甚至忽略

...

\* \*\*`harpoon\_turret.gd` 的 `\_on\_target\_reached\_deck` 方法：\*\*

\* 修改為調用 `monster.on\_pulled\_to\_deck()`。

```gdscript

harpoon_turret.gd

func _on_target_reached_deck(target: MonsterBody):

確保目標被正確設置為上船狀態

target.on_pulled_to_deck() # 調用怪物自身處理上船後的邏輯

Global.hooked_monsters.erase(target) # 從全局掛鉤列表中移除

清理魚叉，由魚叉自身發出 retracted_to_launcher 信號，最終調用

complete_retraction

if is_instance_valid(_current_harpoon):

_current_harpoon.start_pullback() # 讓魚叉啟動自動回收程序

...

改造目標 4：完善 `player.gd` 中的交互邏輯

* **`player.gd` (文件):**

* 確保 `\_open\_cabinet\_logic` 在調用 `hud.open\_selection\_menu` 時，傳遞的 `items` 是一個有效的陣列：

```gdscript

# player.gd \_open\_cabinet\_logic 方法

# ...

var items = []

var raw\_items = cabinet.get("available\_items")

if raw\_items is Array: # 確保類型為 Array

items = raw\_items

# ...

if items.size() > 0:

hud.open\_selection\_menu(items, cabinet)

\_is\_menu\_open = true

\_dispatch\_telemetry("UI\_STATE: 開啟櫃子選單")

# ...



...

\* 在 `_handle_long_press_logic`` 和 `_execute_instant_interaction`` 中，``is_in_group("harpoon_launcher")`` 和 ``is_in_group("turret_mount")`` 都應該能正確識別砲台，但由於 ``harpoon_turret.gd`` 中沒有設置 ``add_to_group("turret_mount")`` 或 ``("harpoon_launcher")``，這會導致偵測失效。

\* \*\*解決方案：\*\* 在 ``harpoon_turret.gd`` 的 ``_ready()`` 中加入 ``add_to_group("turret_mount")``。

#### 改造目標 5：`Global.hooked\_monsters` 的管理

\* \*\*`Global.gd` (文件):\*\*

\* 這個列表需要在怪物被鉤住時添加，被釋放或擊敗時移除。

\* \*\*在 ``harpoon_turret.gd`` 中的 ``_fire`` 方法中，當成功鉤住怪物後：\*\*

```gdscript

harpoon_turret.gd `_on_impact(collision)` 方法中

...

if hit_obj is MonsterBody: # 確保是怪物

 _target_body = hit_obj

 _is_active = false

 _is_waiting_recovery = false

 _hit_offset = global_position - _target_body.global_position

通知怪物已被鉤住

 _target_body.set_is_hooked(true)

 Global.hooked_monsters.append(_target_body) # 添加到全局列

表

if launcher:

 launcher._current_harpoon = self

...

* **在 ``harpoon_turret.gd`` 的 ``_on_target_reached_deck`` 或怪物死亡時移除：**

```gdscript

# harpoon\_turret.gd `_on_target_reached_deck` 方法中

# ...

Global.hooked\_monsters.erase(target) # 從全局列表移除

```
target.set_is_hooked(false) # 通知怪物已解除鉤住狀態
...
``
```

以及在怪物被擊殺或從船上掉落時，也應執行  
`Global.hooked\_monsters.erase(self)`。

---

## ## 總結與展望

這次的錯誤分析揭示了在複雜系統開發中，\*\*清晰的組件職責劃分和嚴謹的通信契約\*\*是多麼重要。通過上述改造，我們將實現以下目標：

1. **統一的蒸汽管理：** `Global.gd` 將成為蒸汽壓力的唯一可靠數據源，避免數據混亂。
2. **明確的魚叉-砲台-怪物交互：** 每個組件都將擁有清晰的職責和調用接口，減少模塊間的隱式依賴。砲台負責發射、拉扯和蒸汽消耗，魚叉負責飛行和碰撞，怪物則響應被拉扯和上甲板的事件。
3. **更健壯的程式碼：** 透過強類型檢查、防禦性編程和明確的信號機制，程式碼將更易於理解、維護和擴展。

這些改動將為《餘燼航路：鏽蝕重生》的戰鬥系統奠定更堅實的基礎，讓您能夠在此之上自信地疊加更多創意和深度玩法。期待這些調整能幫助您的團隊順利推進專案！